LTC CARBON · GITHUB COPILOT ENABLEMENT SESSION

Teach Copilot once. Every team benefits.

Skills, Hooks and the ADLC plugin for GitHub Copilot

```
> /adlc:spec "transaction search"_
```

Saiganesh · 30 Oct 2026

LIVE DEMO · 90 SECONDS

Before any slides: watch this

1 · Ask Copilot to touch prod config

```
> add a TODO to application-prod.yml
Copilot → edit application-prod.yml
Hook → permissionDecision: deny
Copilot → "I can't edit that. Want a ticket?"
```

2 · Ask where a feature stands

```
> /adlc:status
transaction-search: spec, bdd approved
design → waiting for your approval
next → approve design, then /adlc:plan
```

Neither of those was Copilot being polite. One was a guarantee. The other was our process. That's today's session.

THE PROBLEM

Every team re-teaches Copilot from scratch

The copy-paste prompt

Someone’s perfect prompt lives in a Teams chat from March. Nobody can find it, so everyone writes their own.

The 800-line instructions file

copilot-instructions.md keeps growing. It applies to every request, whether today’s task needs it or not.

The 95% rule

“Please don’t touch prod config” works most of the time. In a bank, “most of the time” isn’t good enough.

Skills fix the first two. **Hooks** fix the third. A **plugin** ships all of it to every team.

THE MENTAL MODEL

Six pieces, explained as a new joiner

Piece	New-joiner analogy	When it applies	Example at LTC
Instructions	The team handbook	Every request, always	Java 21 · Maven · our conventions
Skill	A playbook on the shelf	When the task matches	/adlc:bdd writes Gherkin
Hook	The door badge	On every lifecycle event	Block edits to prod config
Custom agent	A specialist you call in	When picked or delegated	A read-only design critic
MCP server	The tools on your desk	When Copilot calls a tool	Jira, Confluence, databases
Plugin	The onboarding kit	Installed once, versioned	adlc bundles all of the above

Repo paths: .github/copilot-instructions.md, skills/, hooks/, agents/

PART 1 OF 3

Skills

Team know-how that loads itself, only when the task needs it.

SKILLS · ANATOMY

A skill is a folder with a SKILL.md

```
.github/skills/release-notes/
├── SKILL.md
└── scripts/collect.sh

---
name: release-notes
description: Draft release notes from
  recent commits. Use for release
  notes or a changelog.
---
Group the commits by feature for
the Carbon team.
```

Where it lives decides who gets it

Project · .github/skills/ — commit it and the whole team has it

Personal · ~/.copilot/skills/ — every project on your machine

Plugin · installed from a marketplace, with a version

Portable · Copilot also reads .claude/skills/ and .agents/skills/

The **description** is the trigger. Write it like the words people will actually type.

SKILLS · WHY THEY SCALE

Skills load in three layers

LAYER 1 · ALWAYS

Discover

name + description

Copilot matches your request against every skill's description.

LAYER 2 · WHEN IT MATCHES

Load

the SKILL.md body

The instructions enter context only when the task matches.

LAYER 3 · IF NEEDED

Reach

scripts and files

Templates and references load only when the instructions point to them.

"Copilot loads only what is relevant for each task." A big skill library costs less than one bloated instructions file.

Source: VS Code docs, "Use Agent Skills in VS Code"

SKILLS · HIDDEN SUPERPOWERS

Five skill tricks people miss

`disable-model-invocation: true`	**Only a human** can trigger it, as a slash command. Use it for anything that deploys, ships or sends.
`user-invocable: false`	Hidden from the / menu. Copilot **loads it by itself** when the task matches.
`context: fork`	Runs the skill in a **subagent**, so your main chat stays clean. VS Code, experimental.
`allowed-tools: shell`	Copilot CLI: **no permission prompt** for that tool. Only once you've reviewed the skill's scripts.
`/create-skill · /skills reload`	VS Code **drafts a skill** from a description. The CLI picks up new skills without a restart.

LIVE DEMO 1 · 4 MINUTES · VS CODE AGENT MODE

Build a skill live: /release-notes

1. Create .github/skills/release-notes/SKILL.md
2. Paste the frontmatter and three numbered steps
3. Type /release-notes v2.4 in agent mode
4. Now ask “what changed this sprint?” with no slash at all

The wow moment is step 4: Copilot picks the skill by itself, from the description alone.

```
---
name: release-notes
description: Draft release notes from
  recent commits. Use for release notes,
  changelogs or "what changed".
argument-hint: [version]
---
1. Run git log --oneline -20.
2. Group the commits by feature.
3. Write release notes for the given
   version, for the Carbon team.
```

PART 2 OF 3 · HOOKS

Skills suggest. Hooks guarantee.

A hook is a script Copilot runs at a fixed moment. The model doesn’t get a vote.

HOOKS · THE LIFECYCLE

Where you can plug in

`sessionStart`

A session starts or resumes

Adds context

`userPromptSubmitted`

You send a prompt

Logs and audits

`preToolUse`

Before any tool call runs

Can block

`postToolUse`

After a tool call succeeds

Feeds back

`agentStop`

Copilot finishes a turn

Can block

preToolUse answers with JSON on stdout

```
{ "permissionDecision": "deny",
  "permissionDecisionReason": "…" }
```

Gotcha: preToolUse fails closed

If the script crashes or exits non-zero, the tool call is denied. A timeout falls back to normal permissions.

HOOKS · RECIPES

Five hooks worth stealing on Monday

Event · tools	What it does for you
notification (CLI)	Desktop ping when Copilot needs you. Stop babysitting the terminal.
postToolUse · edit, create	Format every edited file. No more “fix formatting” commits.
preToolUse · edit, create	Block edits to prod config and approved .feature files.
agentStop	Return decision “block” until tests pass; Copilot keeps going.
sessionStart	Inject the sprint goal through additionalContext.

One .github/hooks/ folder serves Copilot CLI, the cloud agent and VS Code (preview). Handlers: command (bash or powershell), http, and prompt on sessionStart.

LIVE DEMO 2 · THE REVEAL · COPILOT CLI

How the opening trick worked

.github/hooks/protect-files.sh

```
#!/bin/bash
IN=$(cat)
TOOL=$(jq -r '.toolName' <<<"$IN")
[[ "$TOOL" =~ ^(edit|create)$ ]] || exit 0
FILE=$(jq -r '.toolArgs|if type=="string"
  then fromjson else . end|.path//""' <<<"$IN")
for p in "application-prod.yml" ".env"; do
  [[ "$FILE" == *"$p"* ]] || continue
  echo '{"permissionDecision":"deny",
  "permissionDecisionReason":"Protected file"}'
  exit 0
done
```

.github/hooks/protect-files.json

```
{ "version": 1,
  "hooks": {
    "preToolUse": [{
      "type": "command",
      "bash": ".github/hooks/protect-files.sh",
      "timeoutSec": 10
    }] } }
```

Copilot sends the tool call as JSON on stdin. **deny** stops the edit, and Copilot reads the reason and changes course.

AUDIENCE QUIZ · SHOUT IT OUT

Instructions, skill, hook or agent?

Shout your answer first. Then I'll reveal it.

1 "We use constructor injection, never field @Autowired."

2 Copilot must never run git push --force.

3 How we write an ADR on the Carbon team.

4 Run the formatter after every single edit.

5 Turn an approved spec into Gherkin scenarios.

6 A reviewer that may only read and search the code.

Instructions

Hook · preToolUse

Skill

Hook · postToolUse

Skill · /adlc:bdd

Custom agent

PART 3 OF 3

The ADLC plugin

Our development lifecycle, packaged once and installed with a few lines of settings.

PLUGINS · THE PACKAGING

A plugin ships it all in one box

```
adlc-plugins/ ← a Git repo = the marketplace
├── .claude-plugin/marketplace.json
└── plugins/adlc/
    ├── .claude-plugin/plugin.json
    ├── skills/
    │   init · spec · bdd · design
    │   plan · build · review · ship
    │   status
    ├── agents/critic.md · reviewer.md
    └── templates/
        spec · design · adr · plan · review
```

Namespaced

Every skill arrives as /adlc:spec, /adlc:plan and so on, so it never clashes with anyone else's.

Versioned

Bump the version in plugin.json. Teammates get it with /plugin marketplace update.

Installed by config

Commit the marketplace and enabledPlugins to .claude/settings.json. Teammates trust the folder and accept the install.

ADLC · THE FLOW

Eight gated phases, one command each

1 **/adlc:init**
Detects Maven, Spring Boot, JUnit
`config.yml`

→

2 **/adlc:spec**
Requirements and acceptance criteria
`spec.md`

→

3 **/adlc:bdd**
Gherkin scenarios from the ACs
`.feature`

→

4 **/adlc:design**
Design + ADRs, critic agent challenges
`design.md + ADR`

5 **/adlc:plan**
Test-first tasks, skeleton first
`plan.md`

→

6 **/adlc:build**
Red, green, refactor per task
`code + tests`

→

7 **/adlc:review**
Reviewer agent checks the change
`review.md`

→

8 **/adlc:ship**
Conventional commit and optional PR
`commit / PR`

Each arrow is your approval. Orange = a subagent reviews. /adlc:status works any time.

What one feature leaves behind

```
.adlc/config.yml ← verify commands
docs/adlc/transaction-search/
  spec.md ← ACs + approvals
  design.md ← design + critic review
  plan.md ← tasks + build log
  review.md ← findings + verdict
docs/adr/0007-use-keyset-pagination.md
src/test/resources/features/
  transaction-search.feature
```

Reviewable

Every decision is a Markdown file in the PR, not a chat that scrolled away.

Traceable

Each task links back to an acceptance criterion and a scenario.

Resumable

Approvals live in spec.md, so anyone can pick up a feature with /adlc:status.

The PR carries its own **why**, right next to the code.

LIVE DEMO 3 · 15 MINUTES

Transaction search, from idea to PR

Time	You type	Watch for
0:00	/adlc:init	Finds Maven, Spring Boot, JUnit by itself
2:00	/adlc:spec "search transactions by date and amount"	Acceptance criteria you can argue with
5:00	/adlc:bdd	Gherkin that maps one-to-one to the ACs
7:00	/adlc:design	The critic agent pushes back on the design
10:00	/adlc:plan, then /adlc:build T1	A failing test first, then green
13:00	/adlc:review, then /adlc:status	PASS or CHANGES REQUESTED, with reasons

Backup plan: a finished run of the same feature on a branch, in case the Wi-Fi or the model is slow.

CLAUDE DESKTOP APP · SHOW AND TELL

Six “wait, it does that?” moments

Parallel worktrees

Three features at once, each in its own Git worktree. Alt-click a session for split view.

Side chat

Cmd/Ctrl + ; or /btw asks a quick question without cluttering the main session.

Browser pane

Claude starts your dev server, clicks through the page and screenshots it to check its own change.

Diff comments

Comment on lines like a PR review. /code-review adds Walk through and Apply fixes.

PR monitoring

CI status per PR, Auto-fix for failing checks and review comments, Merge when ready.

Scheduled tasks

Run a prompt on a schedule, like a weekday-morning summary of overnight PRs.

Not on Team or Enterprise plans: computer use, Dispatch. PR monitoring needs the gh CLI.

WHY THIS IS OURS

The model is shared. The skills are ours.

Our know-how, in Git

Skills encode our standards, templates and approval gates. They live in a repo we own and go through our PR review.

Our own marketplace

The adlc marketplace is a Git repo on our internal server. Nothing goes to a public directory unless we publish it.

Not training data

Under commercial terms, Anthropic doesn't train models on the code or prompts sent to Claude Code.

Anyone can buy the same model. Nobody else has our playbooks.

Source: Claude Code docs, "Data usage" (commercial users)

WHY IT'S SAFE TO ROLL OUT

Platform teams can lock it down

Managed setting	What it gives us
enabledPlugins	Force-install adlc for everyone, or block a plugin everywhere
strictKnownMarketplaces	Allow only the marketplace sources we approve
disableSideloadFlags	No loading plugins from a random folder with --plugin-dir
allowManagedHooksOnly	Limit which hooks run to the ones the org manages
strictPluginOnlyCustomization	Skills, hooks, agents and MCP only from plugins or policy
Enterprise skills folder	Org-deployed skills win over personal and project ones

Same rule as any dependency: a plugin runs with **your** permissions, so we review its source before we approve it.

YOUR MOVE

One skill, one hook, two weeks

1

Install adlc

Commit the marketplace and enabledPlugins to your repo's .claude/settings.json, then run /adlc:init.

2

Write one skill

Pick the thing your team explains most often and turn it into a SKILL.md.

3

Add one hook

Pick the rule you're tired of repeating and make it exit 2.

Skill-a-thon: share yours by [13 Nov]. The best ones go into the shared marketplace for every team.

THANK YOU

Questions?

Skills
code.claude.com/docs/en/skills

Hooks
code.claude.com/docs/en/hooks-guide

Plugins
code.claude.com/docs/en/plugins/overview

The adlc plugin
[internal Git URL] · ask Saiganesh